Text Classification using Graph Neural Networks

Güneykan Özgül
Berk Sudan
Onur Kılıçoğlu

Inzva AI Projects #4 - Showcase - 30.05.2020

Our Team

Güneykan Özgül, Boston Univ. Physics Graduate Student,

R&D Engineer at Pointr: Deep Location Company ozgul@bu.edu guneykan.ozgul@pointrlabs.com

Berk Sudan, YTU Computer Engineering Alumni, Big Data Engineer @TÜBİTAK, berk.sudan.96@gmail.com

Onur Kılıçoğlu, BOUN Computer Engineering Student, onur.kilicoglu@boun.edu.tr



1.
INTRODUCTION

Text Classification & Representation

Text Classification

- Text classification, a **fundamental problem** in NLP
- Some Text Classification Applications:
  - Document Organization
  - News Filtering
  - Spam Detection
  - Opinion Mining

Text Representation

- Text representation, an essential **intermediate step for classification**
- Traditional methods use hand-crafted features such as sparse lexical features
  - Bag-of-Words
  - N-grams

Deep Learning in Text Representation

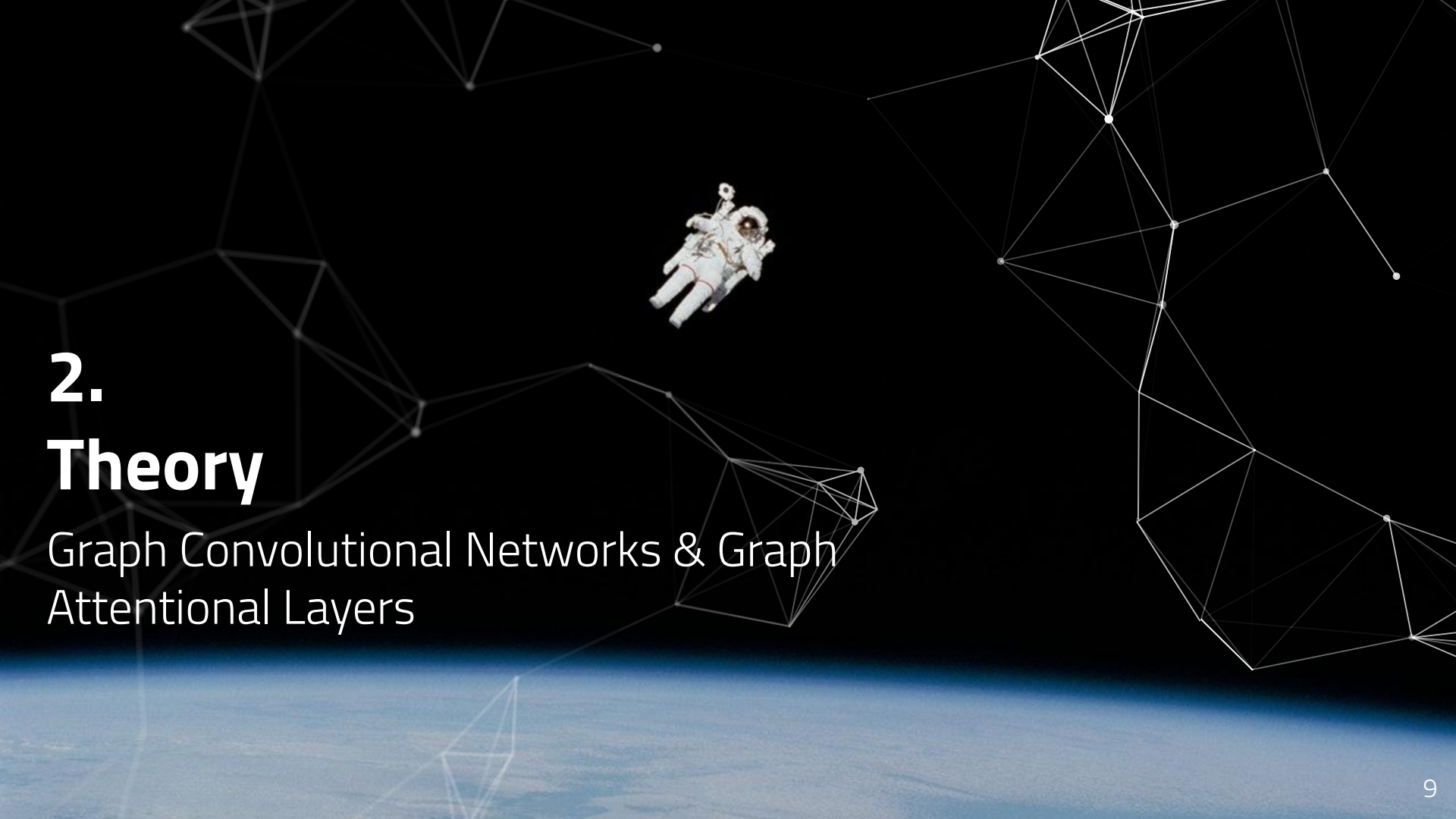
- **Deep learning models** widely used to learn text representations
- **Examples**
  - RNNs such as LSTM (Hochreiter and Schmidhuber 1997)
  - CNNs (Kim 2014)

CNN & RNN in Text Representation

- **Pros**
  - Can capture **semantic** and **syntactic** information in local consecutive word sequences well
- **Cons**
  - May ignore global **word co-occurrence** in a corpus which carries **non-consecutive** and **long-distance semantics** (Peng et al. 2018)

Graph NN in Text Representation

- **Effective at**
  - Tasks thought to have **rich relational structure**
- **Can preserve**
  - **Global structure information** of a graph in graph embeddings



2.
Theory

Graph Convolutional Networks & Graph Attentional Layers

Graph Convolutional Layer - Literature Review

- **Graph Convolutional Networks for Text Classification (Liang Yao, Chengsheng Mao, Yuan Luo, 2019)**
  - A GCN that works directly on a **graphical representation** of a document
  - Induces embedding vectors of nodes based on properties of their **neighborhoods**

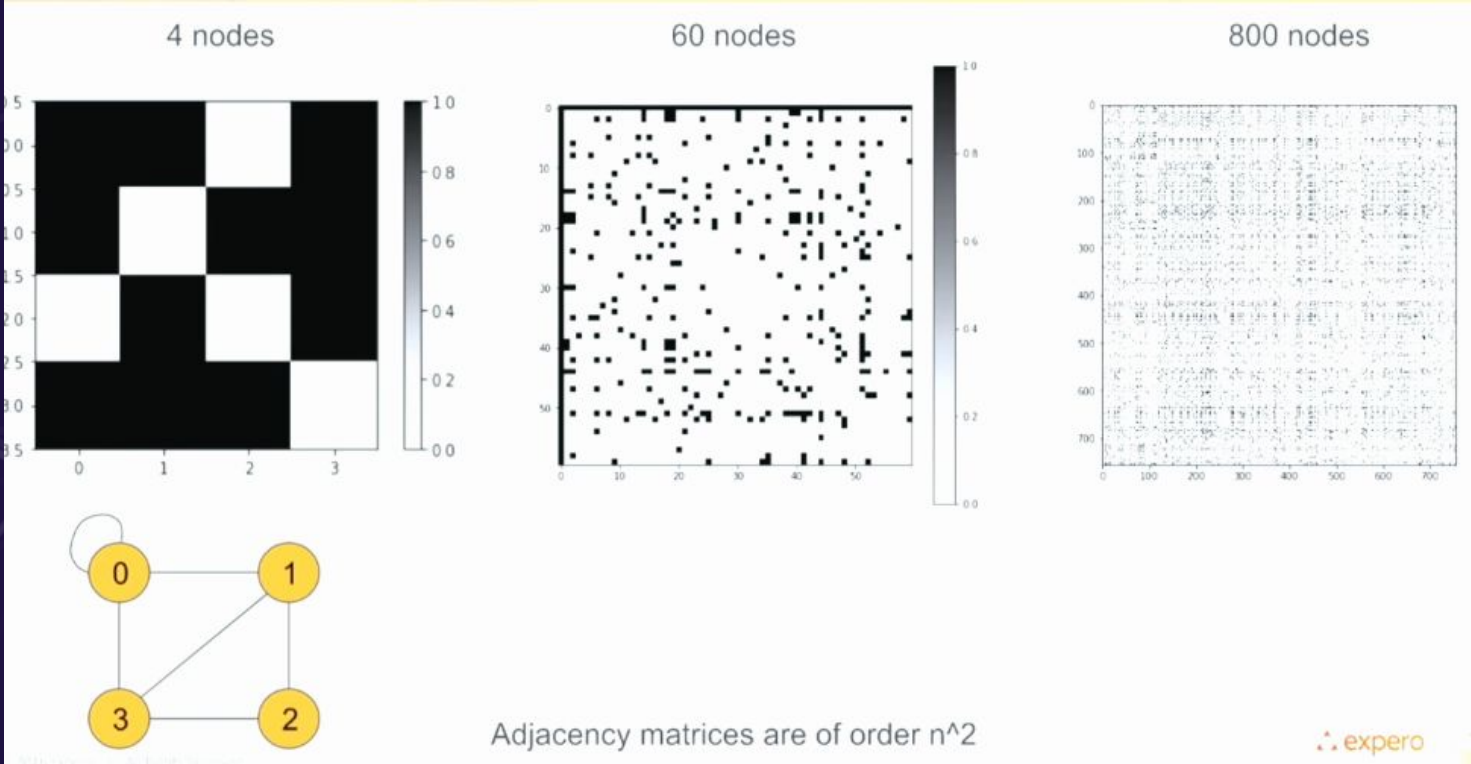
Conventional CNN

- A convolution = mixing operation
- Fixed-size kernels
- Conventional CNN is popular in
  - 1-dimensional time-series data (e.g., sound)
  - 2-dimensional data with spatial locality (e.g., image)
  - 3-dimensional data with spatial locality (e.g., Magnetic Resonance Imaging data)

Graph Convolutional Layer

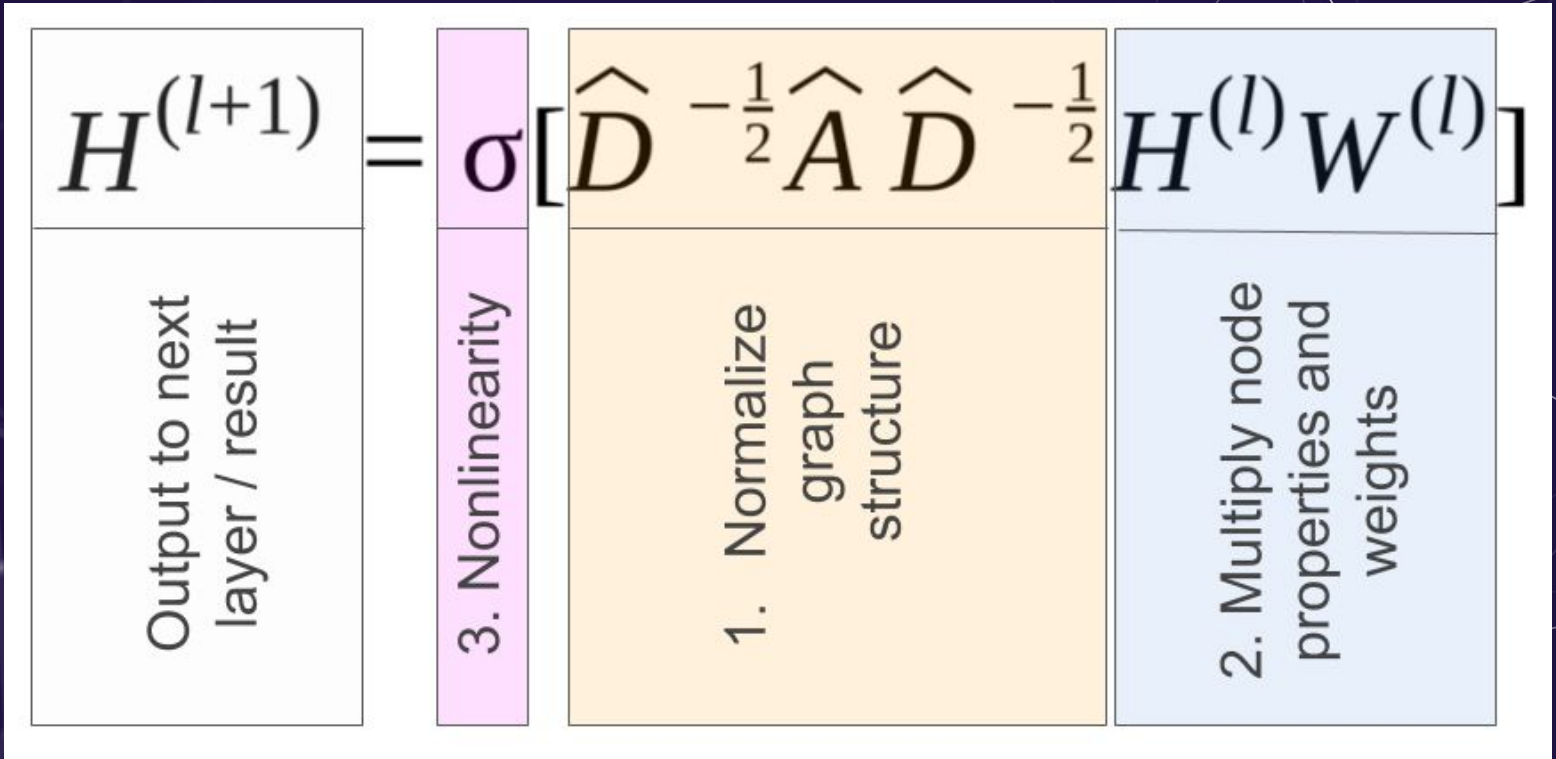
- Number of neighbours changes, so **no fixed-size kernels**
- No global location, but **relative** locations
- Cannot define with **matrices** directly

Locality Problem of Graph-CNN



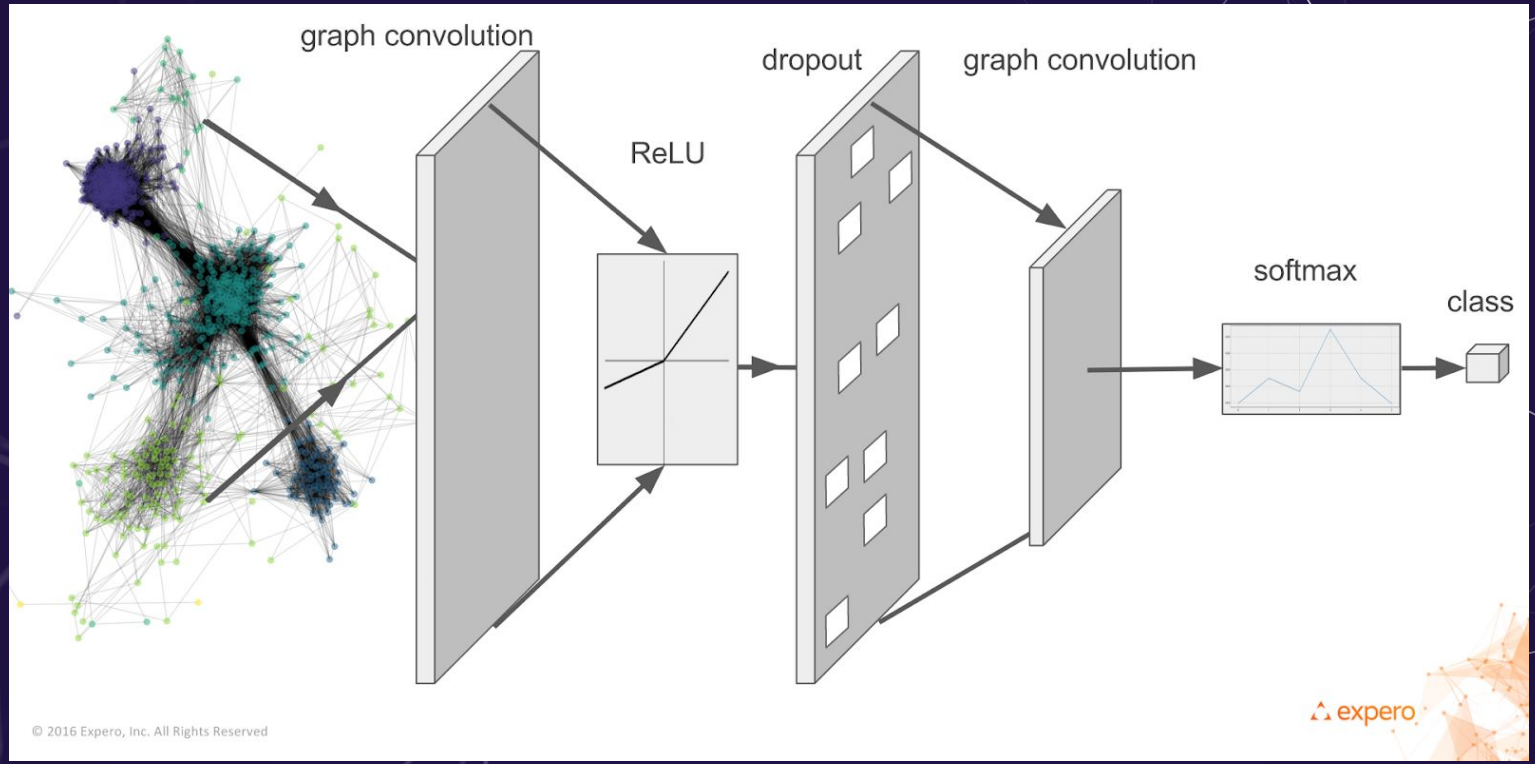
Ref: https://www.experoinc.com/post/node-classification-by-graph-convolutional-network

Graph Convolutional Layer - Formula



Ref: https://www.experoinc.com/post/node-classification-by-graph-convolutional-network

Graph Convolutional Layer - Visually



Ref: https://www.experoinc.com/post/node-classification-by-graph-convolutional-network

Graph Attentional Layer - Literature Review

- **Graph Attention Networks (Velickovic, 2017)**
  - A novel **highly effective** neural network architecture
  - Enables specifying **different weights** to **different nodes** in a neighbourhood

Graph Attentional Layer - Basic Idea

- Can improve the accuracy with an **attention mechanism**

- With attention heads & vectors, **update** node-features

Graph Attentional Layer - Visually

- Self-Attention head

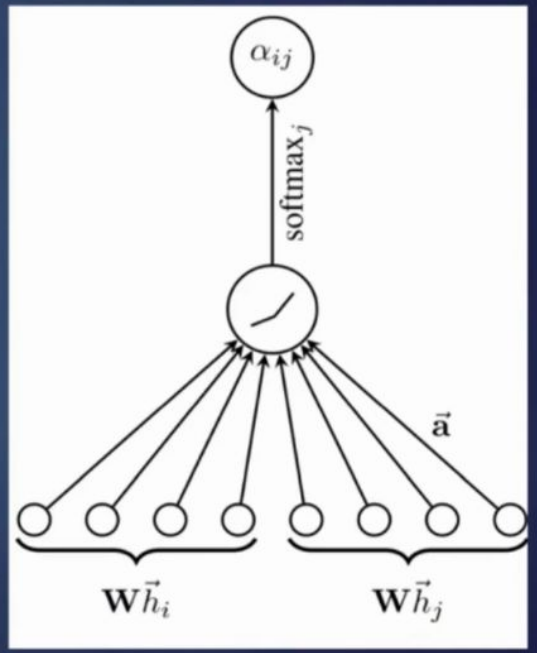
$$e_{ij} = \text{LeakyReLU}\left(\vec{a}^T \left[\boldsymbol{W}\vec{h}_i \| \boldsymbol{W}\vec{h}_j\right]\right)$$

- Output of self-attention head

$$\alpha_{ij} = \text{softmax}_j(e_{ij})$$

- Weights can then be used to compute output

$$\vec{h'}_i = \sigma\left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} \boldsymbol{W}\vec{h}_j\right)$$



Ref: https://ai.science/e/gat-graph-attention-networks--2019-04-15



3.
Our Methods

Our-GCN & Our-GAT

Our GCN Model

- We've implemented a GCN model
  - to learn **word and document embeddings** simultaneously
- A single large graph from an entire corpus, which contains **words and documents** as **nodes**
- A 2-layer GCN can allow message passing among nodes that are at **maximum two steps away**

Our GCN Model - Details

- **No direct document-document edges** in the graph

- The two-layer GCN allows the **information exchange** between pairs of documents

- The second layer node (word/doc) embeddings have the same size as the labels set and are fed into a **softmax classifier**

Our GCN Model - Nodes & Edges

- Edge between two word nodes is built using
  - **Word co-occurrence information (PMI)**
- Edge between a word node and document node is built using
  - **Word frequency and word's document frequency (TF-IDF)**

Our GCN Model - Nodes & Edges

- Introduction of edges between two documents does not improve performance greatly
- Doc-doc edges were built by using hand-crafted metrics (such as cosine similarity) or from GAT output (attention coefficients)
- Initializing the features with one-hot representation (featureless) works equally well

Our GCN Model - Visually

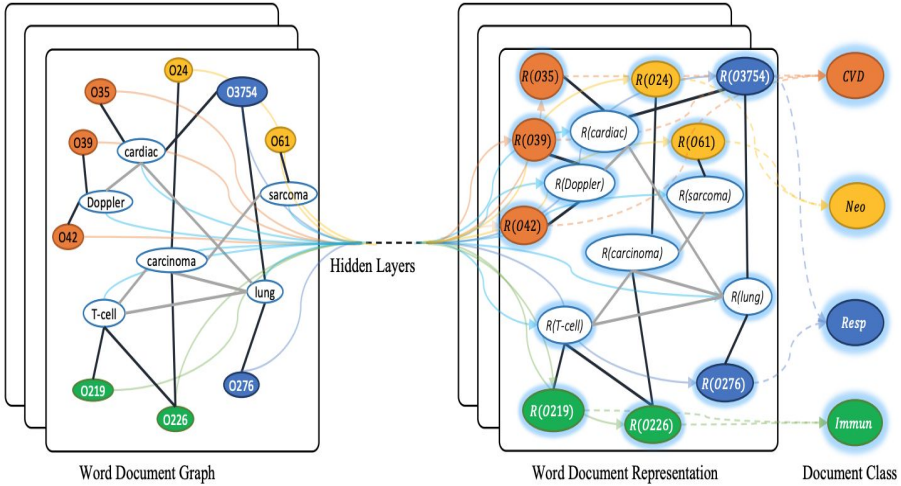


Figure 1: Schematic of Text GCN. Example taken from Ohsumed corpus. Nodes begin with "O" are document nodes, others are word nodes. Black bold edges are document-word edges and gray thin edges are word-word edges. $R(x)$ means the representation (embedding) of x. Different colors mean different document classes (only four example classes are shown to avoid clutter). CVD: Cardiovascular Diseases, Neo: Neoplasms, Resp: Respiratory Tract Diseases, Immun: Immunologic Diseases.

Our GAT

- Operate on **graph-structured data**

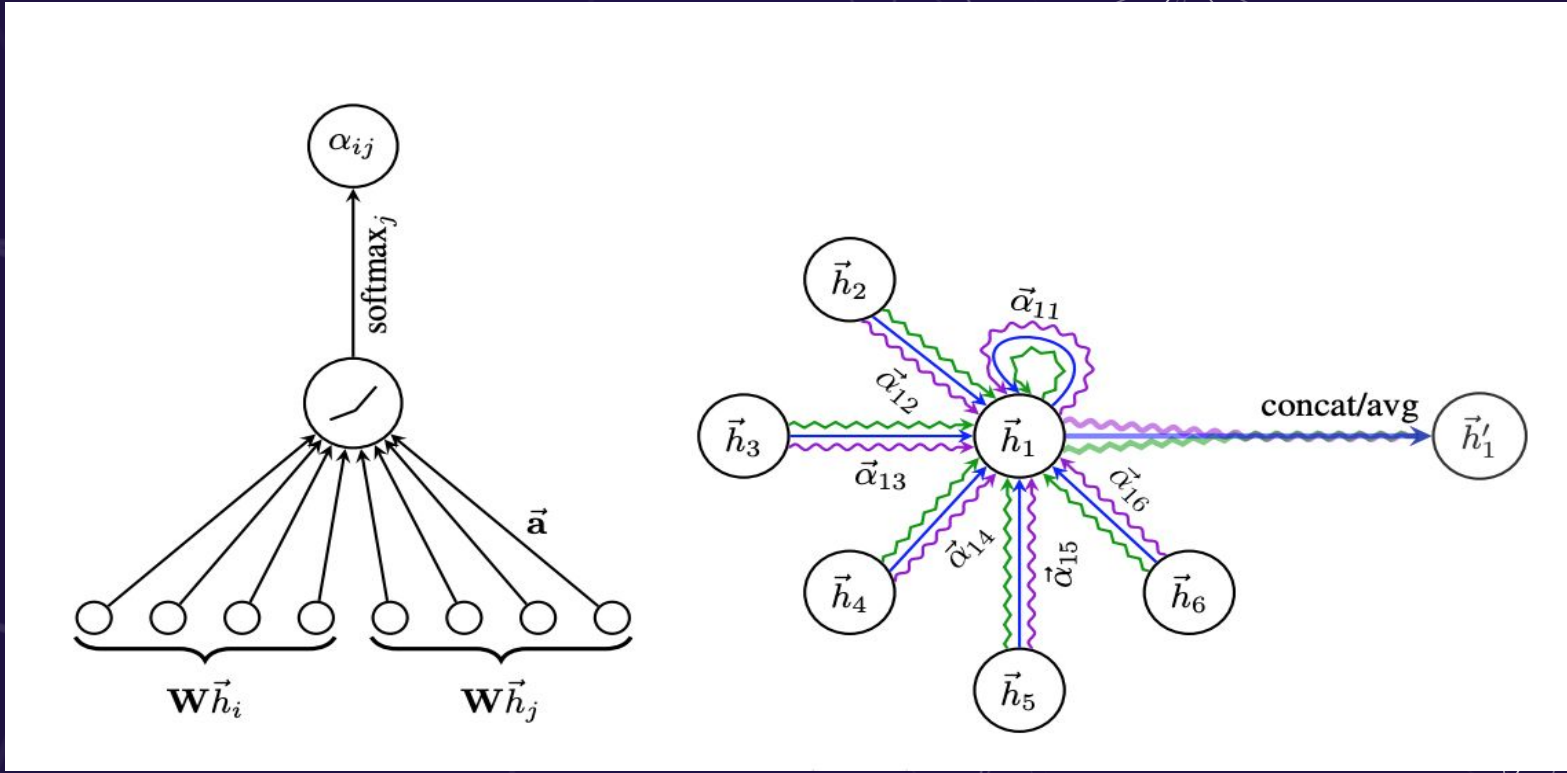
- Leveraging masked **self-attentional layers** to address the shortcomings of prior methods based on graph convolutions (or their approximations)

Our GAT - Details

- By stacking layers in which nodes are able to **attend over their neighborhoods' features**

- Enable (implicitly) specifying different weights to different nodes in a neighborhood
  - without requiring **any costly matrix operation** (such as inversion) or depending on knowing the graph structure upfront

Our GAT - Visually





4.
Evaluation

Data Preparation & Applying GCN/GAT

GCN - Evaluation

- **Pre-Processing**
  - Cleaned Data
  - Removed Rare & Stop Words
  - Shuffled Data (to reduce bias)
- **Building Graph**
  - For Node Features: Training-test docs, word features
  - For Adjacency Matrix: PMI & TF-IDF scores

GAT - Evaluation

- Implemented GAT network to learn **attention coefficients** to be able to specify **different weights for different nodes in neighbourhood**
- Since we do **not have graph data for each dataset**, we only have results for cora, citeseer and pubmed data for now.
- Currently evaluated results on citation graphs

GCN + GAT

- Implemented GAT architecture that works on a graph consisting of words and documents rather than only documents
- Will use attention coefficients to learn graph representation on which GCN operates
- Currently requires some optimization and modifications to have consistent increase in accuracy



5.
Results

Datasets & Results

Our Datasets

20NG

Newsgroups dataset, 18846 documents, 20 labels. Training-Size: 11314, Test-Size: 7532

Ohsumed

Cardiovascular diseases abstracts dataset, 7400 documents, 23 labels. Training-Size: 3357, Test-Size: 4043

R8

Subset of Reuters 21578 dataset, 7674 documents, 8 labels. Training-Size: 5485, Test-Size: 2189

R52

Subset of Reuters 21578 dataset, 9100 documents, 52 labels. Training-Size: 6532, Test-Size: 2568

MR

Movie review dataset, 10662 documents, 2 labels. Training-Size: 7108, Test-Size: 3554

Cora

Citation dataset, 2708 nodes, 5429 edges, 7 classes. Training-Size: 1707, Validation-Size: 189, Test-Size: 812

Citeseer

Citation dataset, 3327 nodes, 4732 edges, 6 classes. Training-Size: 2097, Validation-Size: 232, Test-Size: 998

Pubmed

Citation dataset, 19717 nodes, 44338 edges, 3 classes. Training-Size: 12422, Validation-Size: 1380, Test-Size: 5915

Evaluation Results for Citation Datasets

	Cora	Citeseer	Pubmed
GCN (Kipf, Welling)	**81.5 %**	**70.3 %**	**79.0 %**
Text-GCN (Yao, Mao, Luo)	**75.86 %**	**73.21 %**	**85.87 %**
GAT (Velickovic)	**83.0 ± 0.7 %**	**72.5 ± 0.7 %**	**79.0 ± 0.3 %**

Evaluation Results for Other Datasets

Model	20NG	R8	R52	Ohsumed	MR
TF-IDF + LR	0.8319 ± 0.0000	0.9374 ± 0.0000	0.8695 ± 0.0000	0.5466 ± 0.0000	0.7459 ± 0.0000
CNN-rand	0.7693 ± 0.0061	0.9402 ± 0.0057	0.8537 ± 0.0047	0.4387 ± 0.0100	0.7498 ± 0.0070
CNN-non-static	0.8215 ± 0.0052	0.9571 ± 0.0052	0.8759 ± 0.0048	0.5844 ± 0.0106	**0.7775 ± 0.0072**
LSTM	0.6571 ± 0.0152	0.9368 ± 0.0082	0.8554 ± 0.0113	0.4113 ± 0.0117	0.7506 ± 0.0044
LSTM (pretrain)	0.7543 ± 0.0172	0.9609 ± 0.0019	0.9048 ± 0.0086	0.5110 ± 0.0150	0.7733 ± 0.0089
Bi-LSTM	0.7318 ± 0.0185	0.9631 ± 0.0033	0.9054 ± 0.0091	0.4927 ± 0.0107	0.7768 ± 0.0086
PV-DBOW	0.7436 ± 0.0018	0.8587 ± 0.0010	0.7829 ± 0.0011	0.4665 ± 0.0019	0.6109 ± 0.0010
PV-DM	0.5114 ± 0.0022	0.5207 ± 0.0004	0.4492 ± 0.0005	0.2950 ± 0.0007	0.5947 ± 0.0038
PTE	0.7674 ± 0.0029	0.9669 ± 0.0013	0.9071 ± 0.0014	0.5358 ± 0.0029	0.7023 ± 0.0036
fastText	0.7938 ± 0.0030	0.9613 ± 0.0021	0.9281 ± 0.0009	0.5770 ± 0.0049	0.7514 ± 0.0020
fastText (bigrams)	0.7967 ± 0.0029	0.9474 ± 0.0011	0.9099 ± 0.0005	0.5569 ± 0.0039	0.7624 ± 0.0012
SWEM	0.8516 ± 0.0029	0.9532 ± 0.0026	0.9294 ± 0.0024	0.6312 ± 0.0055	0.7665 ± 0.0063
LEAM	0.8191 ± 0.0024	0.9331 ± 0.0024	0.9184 ± 0.0023	0.5858 ± 0.0079	0.7695 ± 0.0045
Graph-CNN-C	0.8142 ± 0.0032	0.9699 ± 0.0012	0.9275 ± 0.0022	0.6386 ± 0.0053	0.7722 ± 0.0027
Graph-CNN-S	–	0.9680 ± 0.0020	0.9274 ± 0.0024	0.6282 ± 0.0037	0.7699 ± 0.0014
Graph-CNN-F	–	0.9689 ± 0.0006	0.9320 ± 0.0004	0.6304 ± 0.0077	0.7674 ± 0.0021
Text GCN	**0.8634 ± 0.0009**	**0.9707 ± 0.0010**	**0.9356 ± 0.0018**	**0.6836 ± 0.0056**	0.7674 ± 0.0020

Code is available at

- https://github.com/guneykan/gcn_text_categorization
- https://github.com/inzva/Text-Classification-Graph-CNNs

References

- [Liang Yao, Chengsheng Mao, Yuan Luo, 2018] Graph Convolutional Networks for Text Classification: https://arxiv.org/abs/1809.05679
- [Kipf and Welling, 2017] Semi-supervised classification with graph convolutional networks. InICLR.
- [Ankit Pal] Multi-label Text Classification using Attention based Graph Neural Network
- [Petar Velickovic] Graph Attention Networks

Text Classification using Graph Convolutional Networks

Thank you for listening!

Güneykan Özgül
Berk Sudan
Onur Kılıçoğlu